WEEK-04
FRACTIONAL KNAPSACK USING GREEDY TECHNIQUE AND LEETCODE: MINIMUM UNITS ON A TRUCK

Start here | knapsack.c

```c
#include <stdio.h>

// Structure for items
struct Item {
    int value;
    int weight;
    float ratio;
};

// Function to sort items by ratio (descending)
void sort(struct Item items[], int n) {
    int i, j;
    struct Item temp;

    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (items[i].ratio < items[j].ratio) {
                temp = items[i];
                items[i] = items[j];
                items[j] = temp;
            }
        }
    }
}

// Function for Fractional Knapsack
float fractionalKnapsack(struct Item items[], int n, int capacity) {
    float totalValue = 0.0;

    // Calculate ratio
    for (int i = 0; i < n; i++) {
        items[i].ratio = (float)items[i].value / items[i].weight;
    }

    // Sort items by ratio
    sort(items, n);

```

```c
    // Greedy selection
    for (int i = 0; i < n; i++) {
        if (capacity >= items[i].weight) {
            totalValue += items[i].value;
            capacity -= items[i].weight;
        } else {
            totalValue += items[i].value * ((float)capacity / items[i].weight);
            break;
        }
    }

    return totalValue;
}

int main() {
    int n, capacity;

    printf("Enter number of items: ");
    scanf("%d", &n);

    struct Item items[n];

    for (int i = 0; i < n; i++) {
        printf("Enter value and weight of item %d: ", i + 1);
        scanf("%d %d", &items[i].value, &items[i].weight);
    }

    printf("Enter capacity of knapsack: ");
    scanf("%d", &capacity);

    float maxValue = fractionalKnapsack(items, n, capacity);

    printf("Maximum value = %.2f\n", maxValue);

    return 0;
}
```

```
Enter number of items: 3
Enter value and weight of item 1: 60 10
Enter value and weight of item 2: 100 20
Enter value and weight of item 3: 120 30
Enter capacity of knapsack: 50
Maximum value = 240.00

Process returned 0 (0x0)   execution time : 64.205 s
Press any key to continue.
```

Problem List

Description | Accepted | Editorial | Solutions | Submissions

1710. Maximum Units on a Truck

Solved

Easy | Topics | Companies | Hint

You are assigned to put some amount of boxes onto **one truck**. You are given a 2D array `boxTypes`, where `boxTypes[i] = [numberOfBoxes$_i$, numberOfUnitsPerBox$_i$]`:

- `numberOfBoxes$_i$` is the number of boxes of type `i`.
- `numberOfUnitsPerBox$_i$` is the number of units in each box of the type `i`.

You are also given an integer `truckSize`, which is the **maximum** number of **boxes** that can be put on the truck. You can choose any boxes to put on the truck as long as the number of boxes does not exceed `truckSize`.

Return *the **maximum** total number of **units** that can be put on the truck.*

Example 1:

```
Input: boxTypes = [[1,3],[2,2],[3,1]], truckSize = 4
Output: 8
Explanation: There are:
- 1 box of the first type that contains 3 units.
- 2 boxes of the second type that contain 2 units each.
- 3 boxes of the third type that contain 1 unit each.
You can take all the boxes of the first and second types, and one box of the third type.
The total number of units will be = (1 * 3) + (2 * 2) + (1 * 1) = 8.
```

Example 2:

```
Input: boxTypes = [[5,10],[2,5],[4,7],[3,9]], truckSize = 10
Output: 91
```

Constraints:

- `1 <= boxTypes.length <= 1000`
- `1 <= numberOfBoxes$_i$, numberOfUnitsPerBox$_i$ <= 1000`
- `1 <= truckSize <= 10^6`

Submit | 0 | Premium

Code

C | Auto

```c
#include <stdio.h>
#include <stdlib.h>

// Comparator for sorting in descending order of units per box
int compare(const void *a, const void *b) {
    int *box1 = *(int **)a;
    int *box2 = *(int **)b;
    return box2[1] - box1[1]; // descending
}

int maximumUnits(int** boxTypes, int boxTypesSize, int* boxTypesColSize, int truckSize) {

    // Step 1: Sort by units per box (descending)
    qsort(boxTypes, boxTypesSize, sizeof(int*), compare);

    int totalUnits = 0;

    // Step 2: Greedy selection
    for (int i = 0; i < boxTypesSize; i++) {
        int boxes = boxTypes[i][0];
        int units = boxTypes[i][1];

        if (truckSize >= boxes) {
            totalUnits += boxes * units;
            truckSize -= boxes;
        } else {
            totalUnits += truckSize * units;
            break;
        }
    }

    return totalUnits;
}
```

Saved | Ln 3, Col 1

SHIRISHA B R

Access all features with our Premium subscription!

My Lists | Notebook | Progress | Points

Try New Features

Orders

My Playgrounds

Settings

Appearance

Sign Out

Problem List

Submit

Description | Accepted | Editorial | Solutions | Submissions

1710. Maximum Units on a Truck

Solved

Easy | Topics | Companies | Hint

You are assigned to put some amount of boxes onto **one truck**. You are given a 2D array `boxTypes`, where `boxTypes[i] = [numberOfBoxes$_i$, numberOfUnitsPerBox$_i$]`:

- `numberOfBoxes$_i$` is the number of boxes of type `i`.
- `numberOfUnitsPerBox$_i$` is the number of units in each box of the type `i`.

You are also given an integer `truckSize`, which is the **maximum** number of **boxes** that can be put on the truck. You can choose any boxes to put on the truck as long as the number of boxes does not exceed `truckSize`.

Return *the **maximum** total number of **units** that can be put on the truck.*

Example 1:

```
Input: boxTypes = [[1,3],[2,2],[3,1]], truckSize = 4
Output: 8
Explanation: There are:
- 1 box of the first type that contains 3 units.
- 2 boxes of the second type that contain 2 units each.
- 3 boxes of the third type that contain 1 unit each.
You can take all the boxes of the first and second types, and one box of the third type.
The total number of units will be = (1 * 3) + (2 * 2) + (1 * 1) = 8.
```

Example 2:

```
Input: boxTypes = [[5,10],[2,5],[4,7],[3,9]], truckSize = 10
Output: 91
```

Constraints:

- `1 <= boxTypes.length <= 1000`
- `1 <= numberOfBoxes$_i$, numberOfUnitsPerBox$_i$ <= 1000`
- `1 <= truckSize <= 10$^6$`

Code

C Auto

```c
#include <stdio.h>
#include <stdlib.h>

// Comparator for sorting in descending order of units per box
int compare(const void *a, const void *b) {
    int *box1 = *(int **)a;
    int *box2 = *(int **)b;
    return box2[1] - box1[1]; // descending
}

int maximumUnits(int** boxTypes, int boxTypesSize, int* boxTypesColSize, int truckSize) {

    // Step 1: Sort by units per box (descending)
    qsort(boxTypes, boxTypesSize, sizeof(int*), compare);

    int totalUnits = 0;

    // Step 2: Greedy selection
    for (int i = 0; i < boxTypesSize; i++) {
        int boxes = boxTypes[i][0];
        int units = boxTypes[i][1];

        if (truckSize >= boxes) {
            totalUnits += boxes * units;
            truckSize -= boxes;
        } else {
            totalUnits += truckSize * units;
            break;
        }
    }

    return totalUnits;
}
```

SHIRISHA B R
Access all features with our Premium subscription!
My Lists
Notebook
Progress
Points
Try New Features
Orders
My Playgrounds
Settings
Appearance
Sign Out

Code

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

boxTypes =

[[1,3],[2,2],[3,1]]

truckSize =

4

Output

8

Expected

8

SHIRISHA B R
Access all features with our Premium subscription!
My Lists
Notebook
Progress
Points
Try New Features
Orders
My Playgrounds
Settings
Appearance
Sign Out

Code

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

Input

boxTypes =

```
[[5,10],[2,5],[4,7],[3,9]]
```

truckSize =

```
10
```

Output

```
91
```

Expected

```
91
```

SHIRISHA B R

Access all features with our Premium subscription!

My Lists | Notebook | Progress

Points

Try New Features

Orders

My Playgrounds

Settings

Appearance

Sign Out